Memoria Proyecto UnLighted

Contornos Inmersivos, Interactivos y de Entretenimiento

Universidade da Coruña

Grupo: 5

Integrantes:

David Dans Villares
Alejandra Gacho Mediavilla
Elena Brezmes Villaraviz

15 de marzo de 2026

Índice general

1 Desarrollo Técnico **3**
1 Videojuego en 2D . . . 3
1.1 Descripción . . . 3
1.1.1 Personajes . . . 3
1.1.2 Enemigos . . . 4
1.1.3 Objetos . . . 4
1.1.4 Diseño . . . 4
1.1.4.1 Guión . . . 4
1.1.4.2 Reglas . . . 4
1.1.4.3 Mecánica . . . 5
1.2 Escenas . . . 5
1.2.1 Metodología de desarrollo y reparto de trabajo. . . . 5
1.2.2 Descripción global de las escenas. . . . 6
1.2.3 Transición entre escenas. . . . 8
1.2.4 Escena 1: TestScene (Escena de juego principal) . . . 9
1.2.4.1 Descripción . . . 9
1.2.4.2 Modelo . . . 9
1.2.4.3 Detalles de implementación . . . 11
1.2.5 Escena 2: MainMenu . . . 11
1.2.5.1 Descripción . . . 11
1.2.5.2 Modelo . . . 12
1.2.5.3 Navegación entre escenas: . . . 12
1.2.5.4 Detalles de implementación . . . 12
1.2.6 Escena 3: DialogScene . . . 13
1.2.6.1 Descripción . . . 13
1.2.6.2 Modelo . . . 13
1.2.6.3 Detalles de implementación . . . 14
1.2.7 Escena 4: PauseScene . . . 14
1.2.7.1 Descripción . . . 14
1.2.7.2 Modelo . . . 14
1.2.7.3 Detalles de implementación . . . 15
1.2.8 Escena 5: GameOverScene . . . 16
1.2.8.1 Descripción . . . 16
1.2.8.2 Modelo . . . 16
1.2.8.3 Detalles de implementación . . . 17
1.2.9 Escena 6: SettingsScene . . . 17
1.2.9.1 Descripción . . . 17
1.2.9.2 Modelo . . . 17
1.2.9.3 Detalles de implementación . . . 18
1.3 Aspectos Destacables . . . 19
1.4 Manual de Usuario . . . 20
1.4.0.1 Requisitos previos. . . . 20
1.4.0.2 Ejecución. . . . 20
1.4.1 Menú principal . . . 20
1.4.2 Controles . . . 21

1.4.3 Mecánicas de juego 21
1.4.4 Guardar y cargar partida 22
1.4.5 Menú de pausa 22
1.4.6 Pantalla de Game Over 23
1.4.7 Ajustes de vídeo 23
1.5 Reporte de *Bugs* 23

Desarrollo Técnico

1. Videojuego en 2D

1.1. Descripción

El objetivo de este apartado es ofrecer una visión general de la estructura y funcionamiento del videojuego UnLighted, sin entrar en detalles técnicos de implementación. Se describen las mecánicas, niveles, controles y reglas generales que definen la experiencia de juego, como base previa a su análisis técnico en secciones posteriores.

- **Tipo de juego y mecánica general:** UnLighted es un videojuego deuego de exploración y sigilo con elementos de puzle en 2D desarrollado con Pygame. El juego está diseñado para un solo jugador y su objetivo principal es, a medida que pasamos las diferentes fases, revertir o solucionar el fallo tecnológico que nos ha llevado a la situación actual. La luz es el aliado del jugador, ya que le permitirá tanto matar enemigos como abrir puertas que llevarán a diferentes lugares.

- **Jugabilidad y estructura por fases:**

  - **Tutorial:** La protagonista empieza el juego en un examen para entrar en la escuela de exploradores, uno de sus compañeros provoca a un Golem que es el desencadenante de toda la historio. A partir de ahí, tiene un profesor que la guia en el primer nivel y después ya intereactua sola.
  - **Nivel 1:** Las ruinas: al enfadarse el Golem, destruye el laboratorio en el que estaban examinandose los alumnos y Nix tiene que recorrerse el laboratorio en busca de objetos,e información para solventar el problema en el que se encuentra.

- **Audio:** El sistema de audio está implementado mediante la clase SoundManager, que encapsula pygame.mixer y diferencia dos tipos de contenido sonoro. Para los efectos de sonido (load_sound / play_sound) utiliza un diccionario en memoria que los carga por nombre desde la ruta de assets definida en config.ini. Para la música de fondo (load_music / play_music) usa el canal dedicado de pygame.mixer.music, con soporte para reproducción en bucle infinito o una sola vez.

1.1.1. Personajes

El personaje principal es Vagalume, una chica nacida de una familia de artesanos, que termina siendo una exploradora. Su funcionalidad se basa en el movimiento y evasión.

Los personajes secundarios son principalmente funcionales o ambientales:

- **Profesor Ako:** Responsable del examne de exploradores , actuará como mentor para el jugador en la introducción del juego.

- **El Joni:**Amigo de la infancia y compañero de Vagalume, es el tonto de la clase. Actuará de alivio humorístico, y será el que desate el desastre que inicia nuestra aventura.

1.1.2. Enemigos

Tenemos dos tipos de enemigos:

- **Sin Luz:** Enemigos sencillos de esquivar, que se encogen y desaparecen al ser iluminados con el rayo concentrado. Algunos solo molestarán y otros bloquearan el camino, limitando la progresion del jugador.
- **Sistemas de defensa del laboratorio:** Enemigos avanzados del juego, su concepto será contrario a los sin luz, habrá que atraerlos a zonas oscuras mientras esquivamos sus ataques para que pierdan la energía y se desactiven.

1.1.3. Objetos

Los objetos que van a estar disponibles durante el juego son:

- **Farol de explorador:** Alumbra el camino del explorador, concentra el rayo para acabar sin luz y activar mecanismos.O apagalo para ocultarte de ciertas amenazas.
- **Flash:** Ataque de área consumible que elimina enemigos.
- **Gancho:** Facilita la bajada por agujeros.
- **Pala:** Herramienta que permite liberar entradas derrumbadas.
- **Interruptor lumínico:** Activa mecanismos al ser iluminado.
- **Puntos de control:** Zonas donde recargar salud, farol y guardar la partida.

1.1.4. Diseño

1.1.4.1. Guión

UnLighted es un juego de exploración y sigilo ambientado en un entorno oscuro. El jugador controla un personaje que debe moverse por un mapa dividido en habitaciones, activar interruptores para desbloquear puertas y avanzar mientras evita o sobrevive a las sombras que lo persiguen. La narrativa se transmite mediante un sistema de diálogos con efecto typewriter, cargados desde ficheros CSV externos, lo que permite añadir texto sin modificar el código.

1.1.4.2. Reglas

El jugador parte con 3 puntos de vida (configurable vía Health(max_hp=3)). Las sombras le persiguen si entran en su rango de visión (ChasePlayer, vision_range=400 píxeles). El contacto con una sombra reduce la vida en 0.5 unidades. Si la vida llega a cero, se activa la GameOverScene. Desde ahí el jugador puede reintentar lo que recarga la última partida guardada o volver al menú principal.

Los interruptores (Switch) se activan pulsando E en un radio de 50 píxeles. Al activarse notifican a sus puertas enlazadas mediante el patrón Observer (SWITCH_ON / SWITCH_OFF), que las desbloquea o bloquea. Las puertas desbloqueadas se abren automáticamente por proximidad al jugador (proximity_range=80 píxeles) y se cierran al alejarse. El movimiento del jugador está limitado por la capa de accesibilidad del mapa

(reachable), definida en Tiled mediante una capa especial que marca casilla a casilla si son transitables.

1.1.4.3. Mecánica

El bucle de juego sigue el esquema clásico events → update → draw a framerate desacoplado mediante dt en milisegundos, lo que hace que la velocidad de movimiento y animaciones sean independientes de los FPS. El renderizado en TestScene se realiza en capas ordenadas: primero el fondo del mapa (map_back, capas con Z <= 0), luego las entidades ordenadas por su coordenada Y inferior (Y-sorting) para simular profundidad, después el mapa de primer plano (map_front, capas con Z >0) que cubre a las entidades, y finalmente el sistema de iluminación. La iluminación funciona renderizando todas las fuentes de luz sobre una superficie negra con BLEND_RGBA_ADD, y multiplicando el resultado sobre la pantalla con BLEND_MULT, produciendo oscuridad fuera de los radios de luz y recortando cada luz al rect de su habitación para evitar que se filtre a través de las paredes.

La progresión entre fases se gestiona mediante el sceneStack: el juego arranca en MainMenu, navega a TestScene al pulsar Play, y desde ahí puede apilar PauseScene, GameOverScene o DialogScene como overlays sin destruir la escena subyacente.

El estado del jugador se persiste mediante SaveManager, que serializa la posición en un fichero JSON. La partida se guarda manualmente con G y se carga automáticamente al iniciar la TestScene.

1.2. Escenas

1.2.1. Metodología de desarrollo y reparto de trabajo.

Durante las primeras semanas de desarrollo, hicimos la división de las tareas que pensabamos que era la correcta, pero nos dimos cuenta de que eran divisiones de gran tamaño y poco específicas, por lo que rehicimos la distribución. A partir de ahí, fuimos trabajando de forma paralela y combinando poco a poco nuestro trabajo. Una vez pasada esta fase de adaptación y distribución, cada miembor del equipo abordaba tareas específicas como: gestión del menú principal, gestíon de la música, animaciones, gestores de recursos... Todo esto, fue desarrollado en **git**, cada miembro trabajaba en una rama, y una vez que tuviese código funcional que cumplía los requisitos de la tarea, hacía merge a la rama principal, en nuestro caso **develop**. El desarrollo no siguió una estrategia incremental ni iterativa basada en versiones funcionales intermedias. Se optó por una estrategia de ensamblado final: se fueron construyendo todos los elementos necesarios para el juego de manera paralela e independiente, y posteriormente se integraron en una versión final funcional.

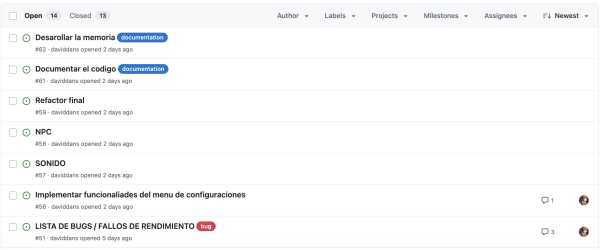
El reparto de tareas fue el siguiente:
→ Implementación del menú principal y estructura del juego: **David**
→ Estructura básica del motor de sonido, para el correcto funcionamiento de la música a lo largo del juego : **Alejandra**
→ Función slide para Tilemap, y así tener listas de tiles accesibles y división de sprites: **Elena**
→ Implementación del gestor de recursos y patrón observador: **Alejandra**
→ Implementación del funcionamiento de la cámara y assets tests **David**

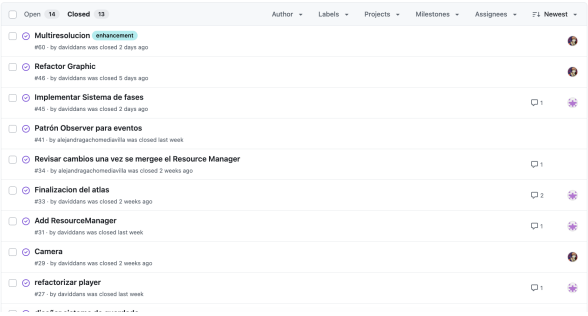
→ Implementación del sistema de guadado **Alejandra**
→ Ventana debug para mostrar información sobre fps, posición del jugador mientras se juega, escena en la que estamos **Elena**
→ Implementación de sistema y aparición de diálogos **Elena**
→ Implementación de graphic refactor y todo el sistema de iluminación **David**

Aunque el trabajo fue mayoritariamente individual y modular, el reparto por niveles permitió que cada integrante tuviera una persona de referencia con la que colaborar directamente cuando surgieran dificultades dentro de su bloque de trabajo.

Gestión de trabajo:

Para gestionar las tareas y ver en que estado se encontraba cada una, empleamos la metodología Kanban. Para ello, como utilizamos git para coordinar todo el trabajo, echamos mano de la opción de Issues dentro de nuestro repositorio, la cuál funciona prácticamente como una tabla Kanban. Tenemos las tareas divididas en Open y Close.





Gestión de versiones con Git:

Utilizamos git como sistema de control de versiones, con un enfoque basado en ramas temáticas. Cada funcionalidad o componente se desarrollaba en una rama independiente, lo que facilitó el trabajo en paralelo y evitó conflictos durante la integración. Los merges, como comentamos antes, se realizaban a la rama principal,en nuestro caso la rama develop, una vez que se completaban las pruebas correspondientes.

1.2.2. Descripción global de las escenas.

El juego tiene 6 escenas en total, organizadas en torno a un sceneStack:

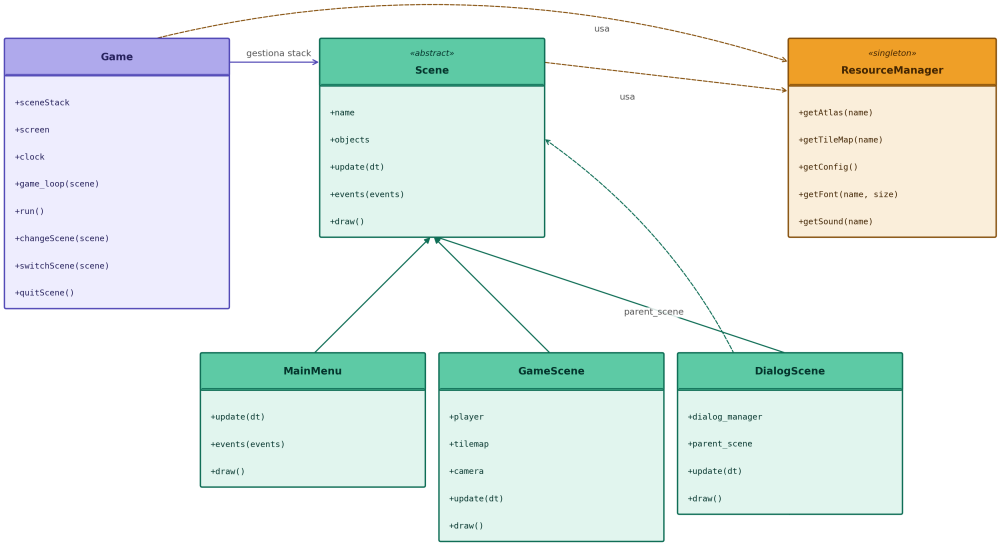
- **MainMenu:** Es el punto de entrada. Tiene tres botones: Play lanza la TestScene (reemplazando el menú), Settings apila la SettingsScene encima, y Quit cierra el juego.

- **TestScene:** Es la escena de juego principal y la más compleja. Carga el mapa desde Tiled, instancia todos los objetos (Player, Shadow, Switch, Door, LightObject) según los metadatos del .tmx, gestiona la cámara con seguimiento del jugador, renderiza por capas con Y-sorting y aplica el sistema de luz con BLEN_DMULT. Al pulsar ESC apila la PauseScene, y si el jugador muere hace switchScene a GameOverScene.

- **DialogScene:** Se apila sobre TestScene como overlay transparente. Muestra un cuadro de diálogo con efecto typewriter cargado desde CSV. Al terminar todos los diálogos hace quitScene y devuelve el control a la escena de debajo.

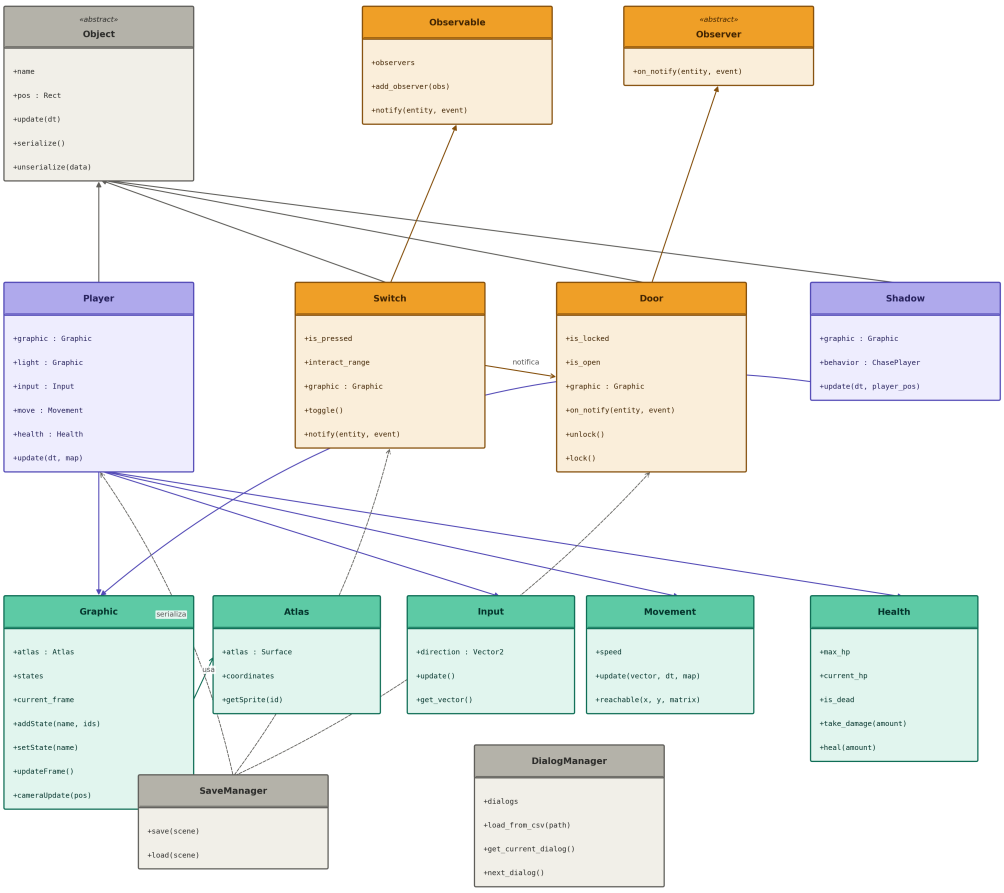
- **PauseScene:** También es un overlay, dibuja la TestScene de fondo con una capa oscura encima. Resume hace quitScene para volver al juego, Settings apila SettingsScene, y Quit navega al menú principal.
- **GameOverScene:** Es otro overlay, esta vez con capa roja. Retry resetea la vida del jugador, recarga el SaveManager y hace quitScene para volver al juego. Main Menu navega al menú principal.
- **SettingsScene:** Permite ajustar resolución, fullscreen y FPS máximos. Los cambios se guardan como pendientes en ResourceManager y se aplican al disco al cerrar el juego. Back hace quitScene.

Diagrama Motor y Escenas:



Motor y escenas — Unlighted

Diagrama Objetos y Componentes:



Objetos y componentes — Unlighted

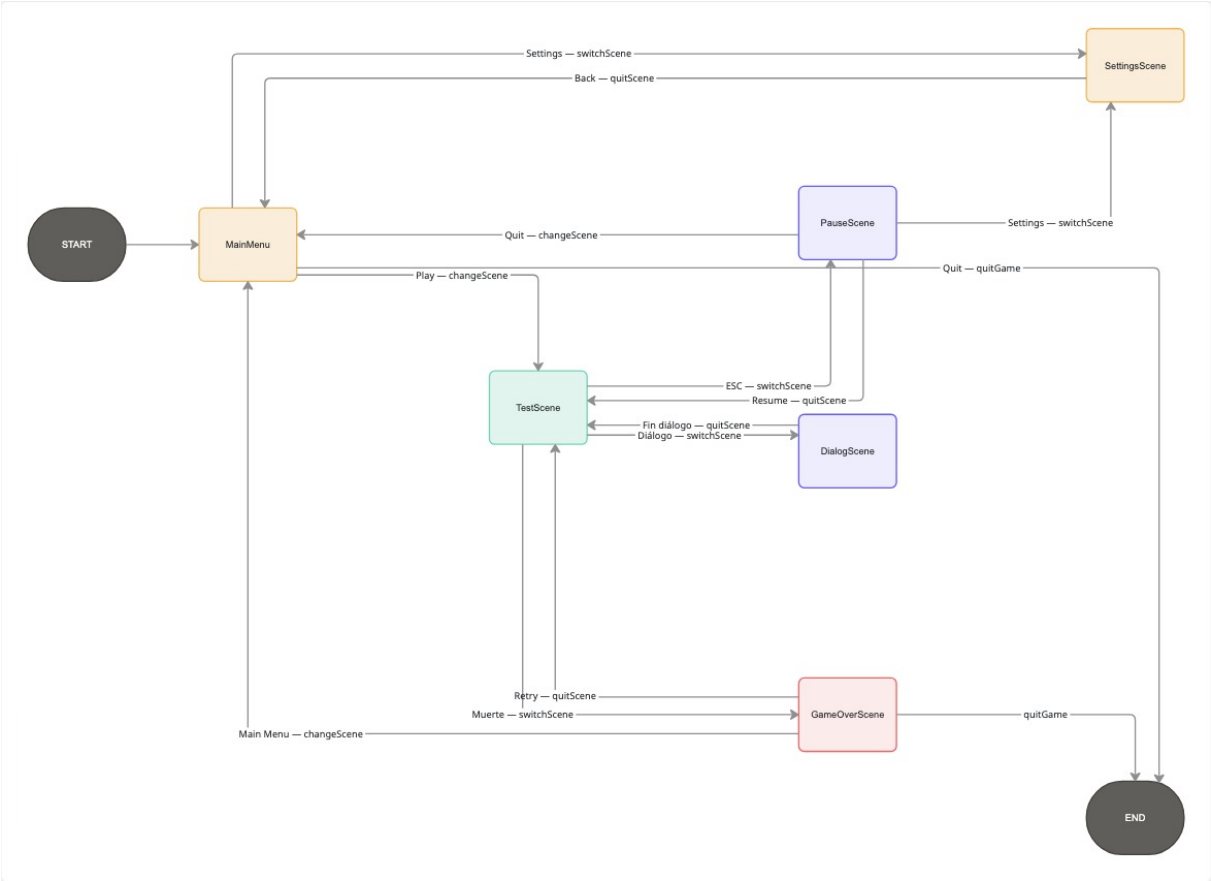
1.2.3. Transición entre escenas.

El sistema usa un sceneStack (lista) y tres operaciones sobre él, cada una con una semántica distinta:

- **changeScene(scene):** Reemplaza sin vuelta atrás. Hace quitScene (que hace pop) y luego push de la nueva escena. Se usa cuando no tiene sentido volver: Play en el menú principal o Quit en la pausa.
- **switchScene(scene):** Apila encima sin destruir la actual. Solo hace push. La escena anterior queda çongelada.en la pila. No recibe update ni events, pero sigue en memoria. Se usa para overlays: ESC → PauseScene, muerte → GameOverScene, diálogo → DialogScene.
- **quitScene():** Desapila y vuelve. Hace pop de la escena actual y activa sceneQuitFlg = True, lo que hace que el game_loop termine su iteración. El bucle run() coge entonces el nuevo tope de la pila y lo ejecuta.

La clave del mecanismo está en sceneQuitFlg: el game_loop no para inmediatamente cuando se llama a quitScene o changeScene, sino que termina el frame actual limpiamente y deja que run() decida cuál es la siguiente escena en el siguiente ciclo. Esto evita que se destruya una escena a mitad de su propio update.

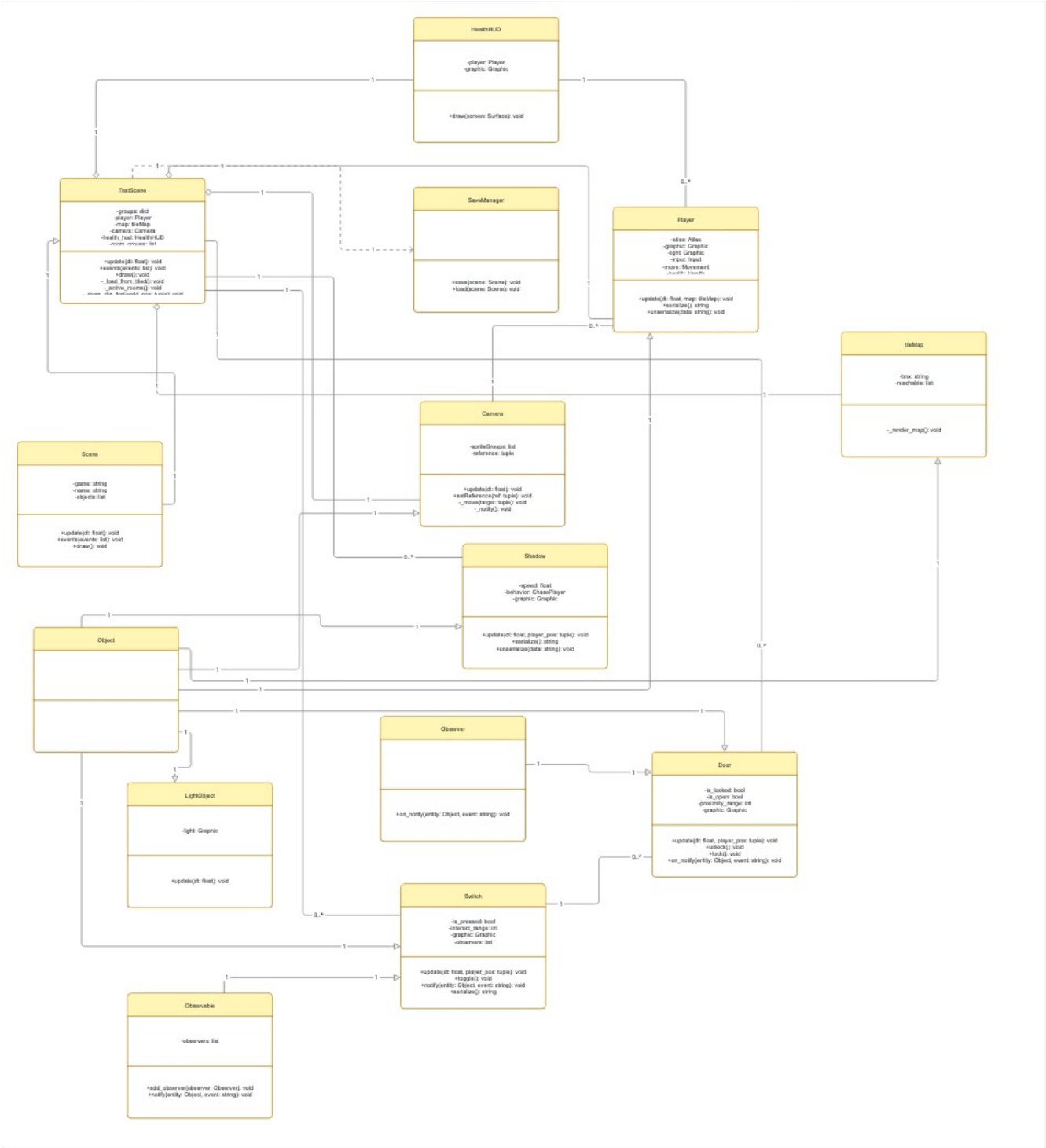
Las escenas overlay (PauseScene, GameOverScene, DialogScene) aprovechan que la escena anterior sigue viva en la pila: en su método draw() llaman explícitamente a self._parent.draw() antes de pintar su propia capa, consiguiendo el efecto de superposición visual sin que el juego subyacente siga actualizándose



1.2.4. Escena 1: TestScene (Escena de juego principal)

1.2.4.1. Descripción `TestScene` es la escena principal de juego. Carga un mapa de tiles desde un archivo `.tmx` (Tiled), instancia todas las entidades definidas en él (jugador, enemigos, puertas, interruptores, luces) y gestiona el bucle de juego: movimiento del jugador, persecución de enemigos, sistema de iluminación por habitaciones y HUD de salud. Si el jugador muere, transiciona a `GameOverScene`; si pulsa Escape, abre el menú de pausa (`PauseScene`).

1.2.4.2. Modelo



Patrones de diseño utilizados:

- **Observer**: `Switch` es `Observable` y notifica a `Door` con eventos `SWITCH_ON` / `SWITCH_OFF`. La puerta reacciona en `on_notify()` sin que el interruptor conozca a la puerta directamente.
- **Component**: `Player` delega su lógica en componentes independientes (`Input`, `Movement`, `Graphic`, `Health`), evitando una clase monolítica.

- **Strategy**: el comportamiento de los enemigos (`ChasePlayer`) es intercambiable gracias a la clase abstracta `abstract.Behavior`, que define `update(npc, dt)`.
- **Scene Stack**: `TestScene` es empujada al stack del motor (`game.sceneStack`) y puede ser suspendida (pausa) o terminada sin destruir el estado subyacente.

Grupos de sprites (sistema de capas):

Grupo	Contenido	Orden de dibujado
`map_back`	Tiles $Z \leq 0$ (suelo)	1º
`entities`	Jugador, enemigos, puertas (`LayeredUpdates`, Y-sort)	2º
`map_front`	Tiles $Z > 0$ (techos)	3º
`lights`	Luces del jugador y estáticas	4º (`BLEND_MULT`)
`hud`	Corazones	5º

Cuadro 1.1: Grupos de sprites en TestScene

1.2.4.3. Detalles de implementación

Carga de entidades desde Tiled: En lugar de instanciar objetos a mano, `_load_from_tiled()` itera los objetos del `.tmx` y los instancia dinámicamente usando un diccionario de clases:

```
entity_classes = {
    "Switch": Switch, "Door": Door,
    "Player": player.Player, "Shadow": Shadow,
}
cls = entity_classes.get(obj_type)
ent = cls(pos=(obj.x, obj.y), graphic_group=..., **obj.properties)
```

Esto permite añadir nuevos tipos de entidad sin modificar el código de la escena.

Vinculación Observer desde Tiled: Tras instanciar todas las entidades, se resuelven las referencias por nombre para conectar switches con puertas:

```
for name in str(ent.target).split(","):
    r = temp.get(name.strip())
    if r:
        ent.add_observer(r)
```

Culling de entidades por room: Solo se actualizan las entidades cuya room solapa con el viewport de la cámara, evitando procesar todo el mapa cada frame:

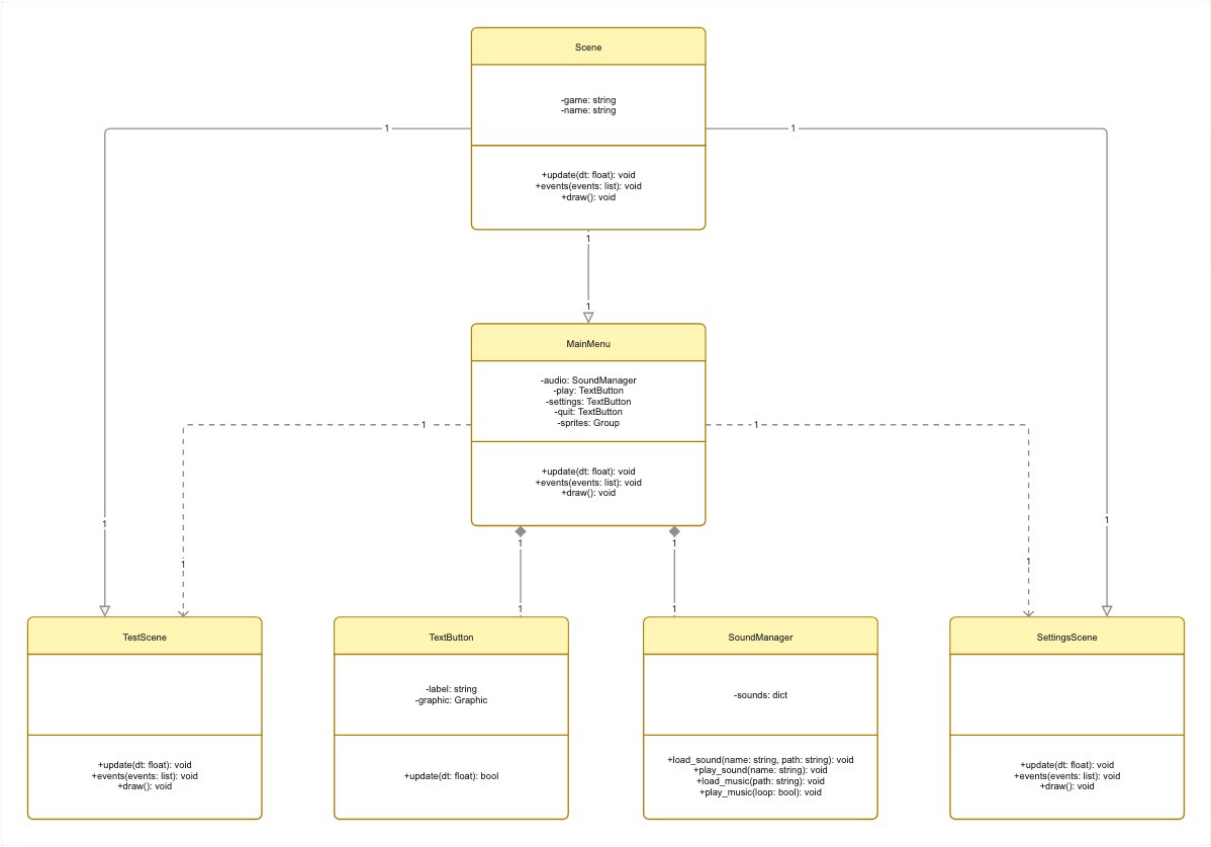
```
def _active_rooms(self):
    return [rect for rects in self.room_groups
            for rect in rects if rect.colliderect(self.camera.pos)]
```

1.2.5. Escena 2: MainMenu

1.2.5.1. Descripción

Escena de entrada al juego. Muestra tres botones (`Play`, `Settings`, `Quit`) sobre fondo blanco. Es la primera escena que se apila en `game.sceneStack` al arrancar el motor. Desde aquí se puede iniciar partida (`changeScene` → `TestScene`), acceder a ajustes de vídeo (`switchScene` → `SettingsScene`) o cerrar la aplicación.

1.2.5.2. Modelo



1.2.5.3. Navegación entre escenas:

- `Play` → `game.changeScene(TestScene)` — reemplaza la escena actual (sin retorno al menú por el stack)
- `Settings` → `game.switchScene(SettingsScene)` — apila la nueva escena; se vuelve al menú con `quitScene`
- `Quit` → `game.quitGame()` — guarda config y limpia el stack

1.2.5.4. Detalles de implementación

Diferencia entre `changeScene` y `switchScene`: Decisión de diseño clave para el stack:

```
def changeScene(self, scene):   # sin retorno
    self.quitScene()
    self.sceneStack.append(scene)

```

```
def switchScene(self, scene):   # con retorno (pausa, settings)
    self.sceneStack.append(scene)
```

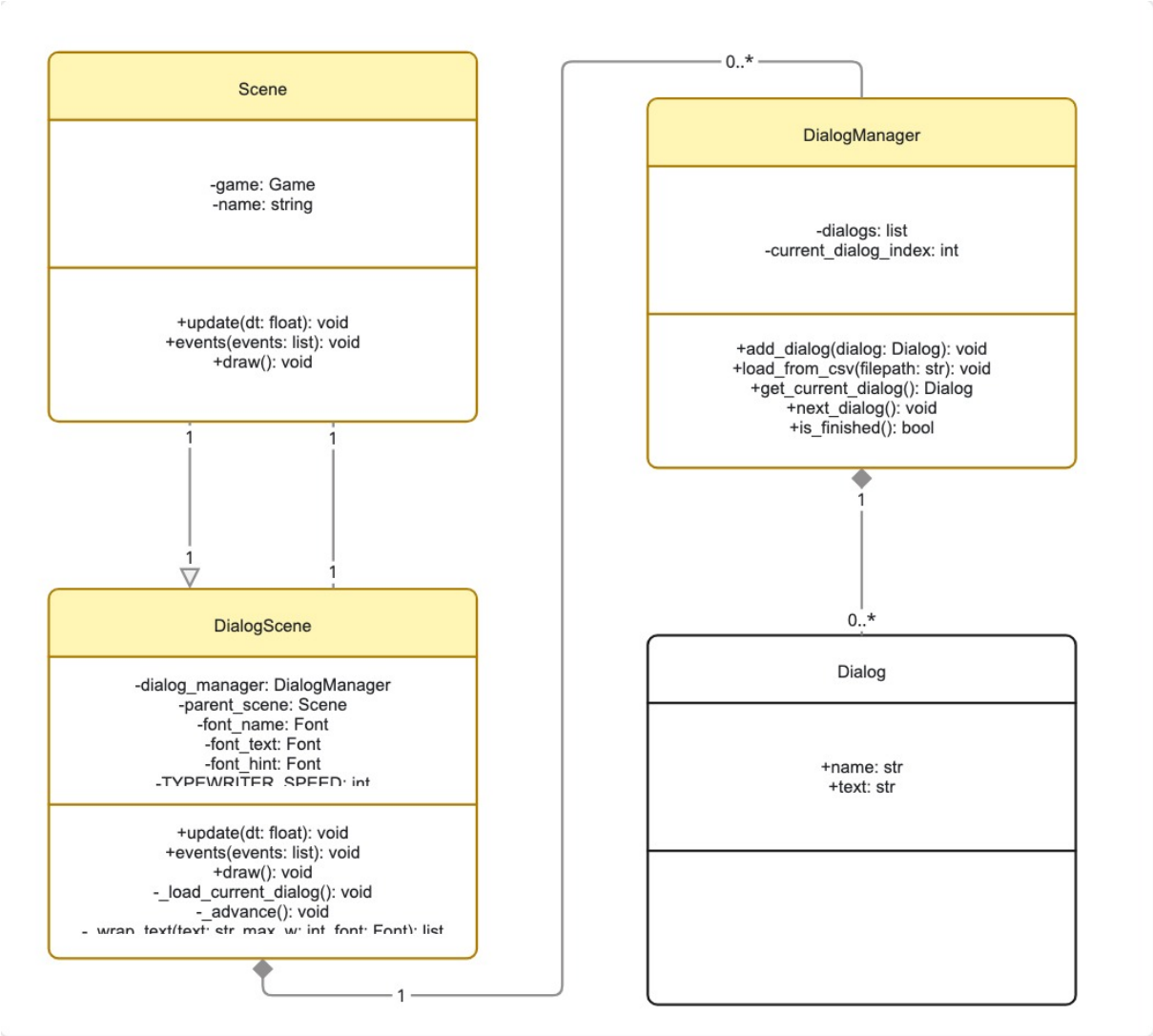
`changeScene` elimina la escena actual antes de empujar la nueva, mientras que `switchScene` la mantiene viva en el stack para poder volver a ella.

1.2.6. Escena 3: DialogScene

1.2.6.1. Descripción

Escena overlay que se apila sobre la escena de juego activa para mostrar diálogos con efecto *typewriter*. Lee su contenido de un `DialogManager` cargado desde CSV. El jugador avanza los diálogos con Espacio o Enter. Al terminar todos los diálogos, se llama a `game.quitScene()` y se retoma la escena subyacente sin interrumpir su estado.

1.2.6.2. Modelo



Flujo de estados del typewriter:

```
_load_current_dialog()
        |
        v
  update(): incrementa _visible_chars segun dt
        |
        v
  _typewriter_done = True  -->  muestra indicador parpadeante
        |
        v
  _advance(): next_dialog() o quitScene()
```

1.2.6.3. Detalles de implementación

Dibujado en capas: `DialogScene` no borra la pantalla, sino que pide a la escena subyacente que dibuje primero, y luego superpone la caja de diálogo. Esto requiere guardar la referencia a `parent_scene` en la construcción:

```
self.parent_scene = game.sceneStack[-1] if game.sceneStack else None

def draw(self):
    if self.parent_scene:
        self.parent_scene.draw()  # dibuja el juego detras
    # luego dibuja la caja encima
```

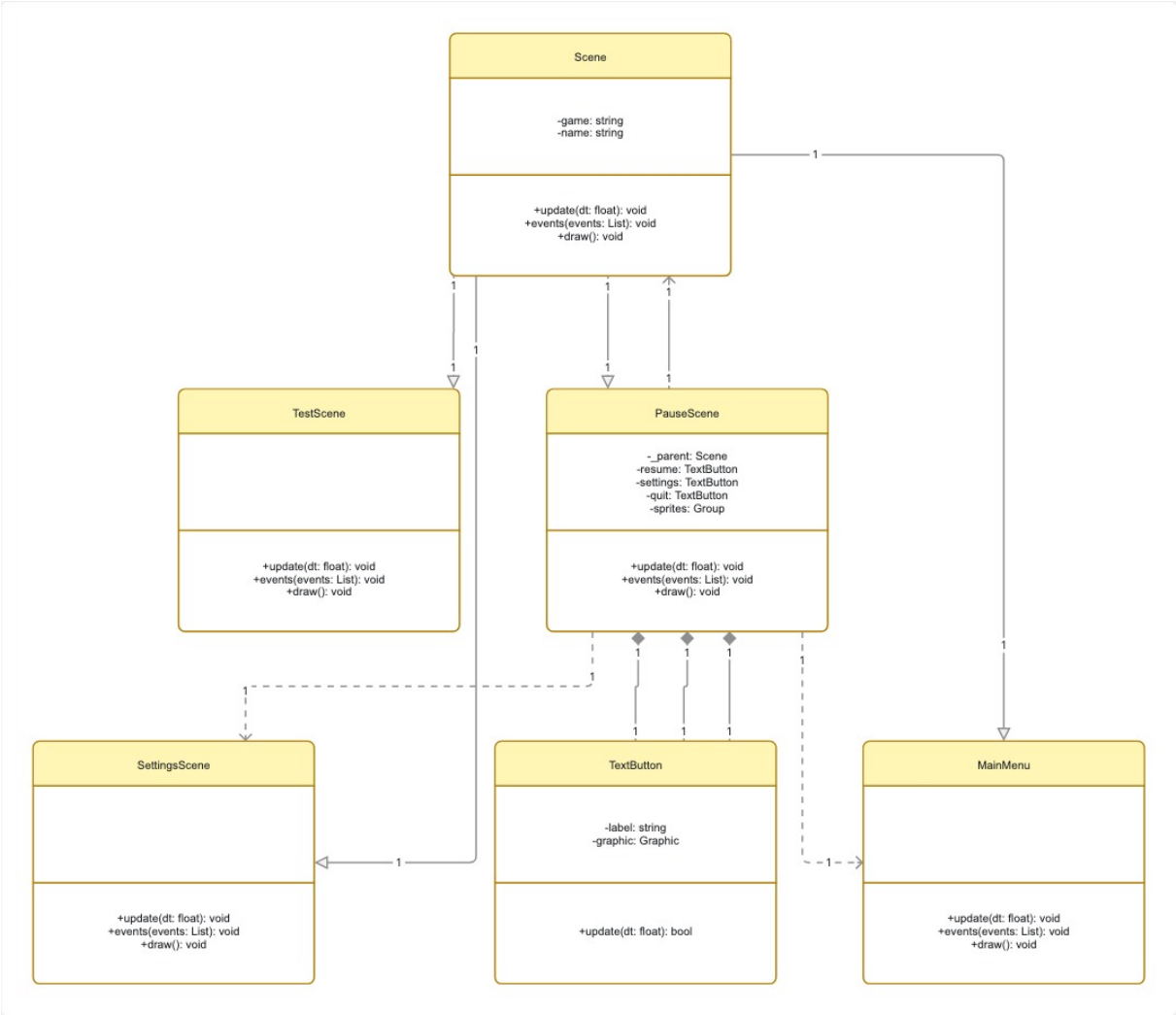
Velocidad typewriter configurable: La constante `TYPEWRITER_SPEED = 40` define los milisegundos por carácter, haciendo trivial ajustar la velocidad sin tocar la lógica de `update`.

1.2.7. Escena 4: PauseScene

1.2.7.1. Descripción

Escena overlay que se apila sobre `TestScene` al pulsar `Escape`. Muestra tres botones (`Resume`, `Settings`, `Quit`) sobre una capa negra semitransparente que oscurece el juego subyacente, el cual permanece congelado en memoria pero visible como fondo.

1.2.7.2. Modelo



Navegación entre escenas:

- `Resume` → `game.quitScene()` — desapila la pausa y devuelve el control a `TestScene`
- `Settings` → `game.switchScene(SettingsScene)` — apila los ajustes encima de la pausa
- `Quit` → `game.changeScene(MainMenu)` — abandona la partida y reemplaza toda la pila con el menú principal

1.2.7.3. Detalles de implementación

Referencia a la escena padre: La referencia se captura en `__init__` antes de que la propia `PauseScene` sea empujada al stack, garantizando que apunta siempre a `TestScene` y no a sí misma:

```
self._parent = game.sceneStack[-1] if game.sceneStack else None
```

Efecto overlay: En `draw()` se renderiza primero la escena padre y después una superficie RGBA semiopaca, consiguiendo la superposición visual sin detener el juego:

```
def draw(self):
    if self._parent:
        self._parent.draw()
    ov = pygame.Surface(self.game.screen.get_size(), pygame.SRCALPHA)
    ov.fill((0, 0, 0, 120))
    self.game.screen.blit(ov, (0, 0))
    self.sprites.draw(self.game.screen)
```

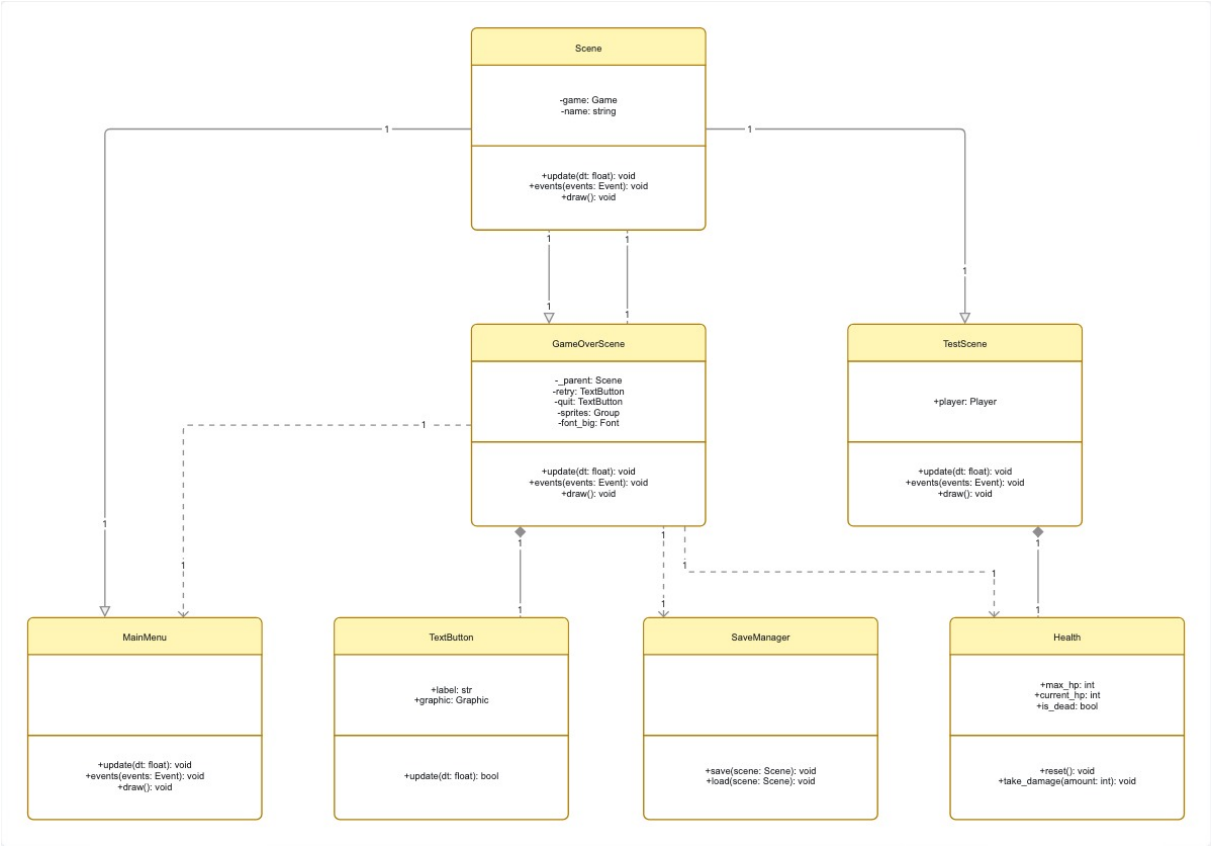
Cierre con `Escape`: Además de los botones, pulsar `Escape` de nuevo en el menú de pausa llama a `quitScene()` directamente desde `events()`, replicando el comportamiento del botón `Resume` sin necesidad de código adicional.

1.2.8. Escena 5: GameOverScene

1.2.8.1. Descripción

Escena overlay que se apila cuando la salud del jugador llega a cero. Tiñe la pantalla de rojo oscuro con una capa semitransparente sobre el estado congelado del juego y muestra el texto "GAME OVER" junto a dos botones: `Retry` y `Main Menu`.

1.2.8.2. Modelo



Navegación entre escenas:

- `Retry` → resetea la salud, recarga el guardado y llama a `game.quitScene()` para volver a `TestScene`
- `Main Menu` → `game.changeScene(MainMenu)` — reemplaza toda la pila con el menú principal

1.2.8.3. Detalles de implementación

Reintentar sin recrear la escena: Al pulsar `Retry`, en lugar de destruir y reconstruir `TestScene` (lo que implicaría recargar el mapa y reinstanciar todas las entidades), se reutiliza directamente la escena padre que permanece en el stack, restaurando únicamente la vida y la posición guardada:

```
if self.retry.update(dt):
    self._parent.player.health.reset()
    SaveManager.load(self._parent)
    self.game.quitScene()  # desapila GameOverScene, vuelve a
       TestScene
```

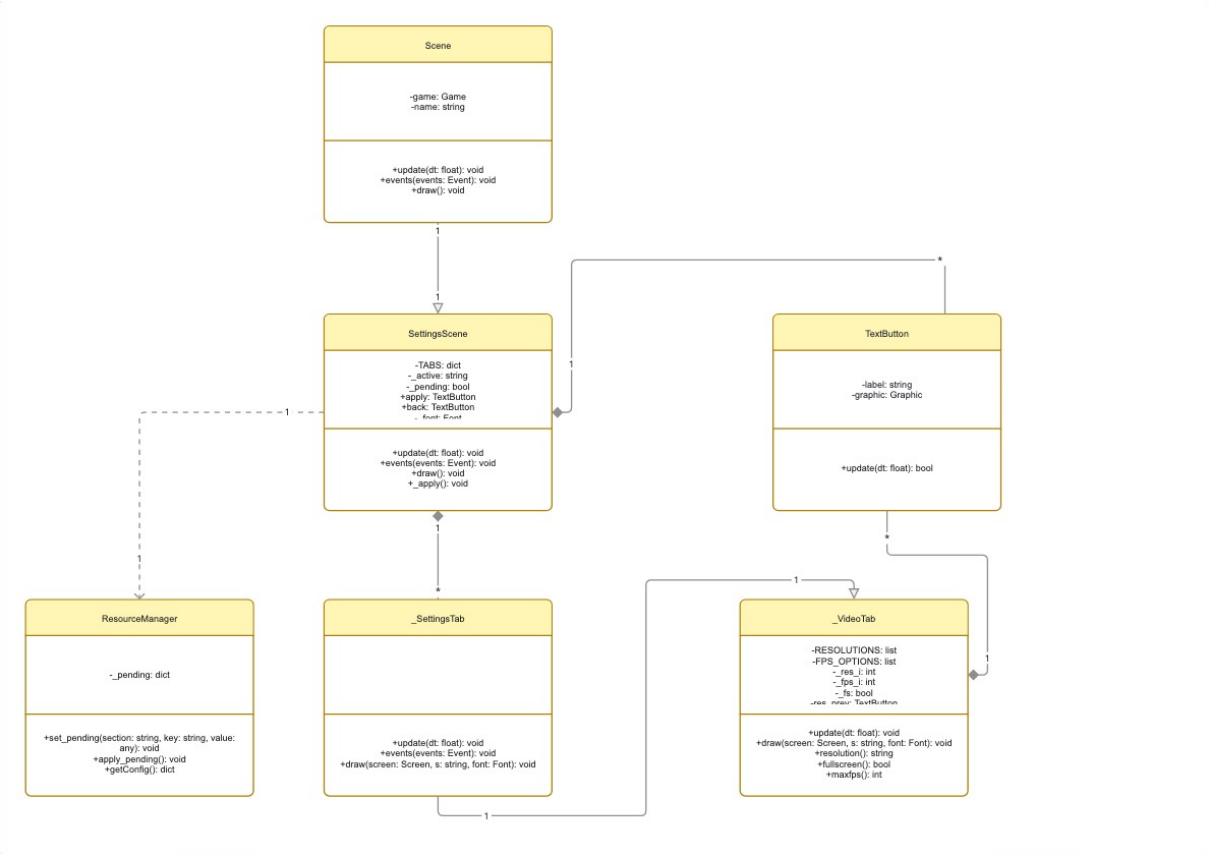
Diferenciación visual respecto a PauseScene: Mientras la pausa usa un overlay negro (`(0, 0, 0, 120)`), el Game Over usa rojo oscuro (`(150, 0, 0, 150)`), comunicando visualmente al jugador el tipo de interrupción sin necesidad de texto adicional.

1.2.9. Escena 6: SettingsScene

1.2.9.1. Descripción

Escena de ajustes de vídeo accesible desde `MainMenu` y desde `PauseScene`. Permite modificar la resolución, el modo de pantalla completa y el límite de FPS. Los cambios no se aplican en caliente sino que se almacenan como pendientes y se escriben al fichero `config.ini` al cerrar el juego.

1.2.9.2. Modelo



Patrones de diseño utilizados:

- **Strategy / Tab:** cada pestaña es una subclase de `_SettingsTab` con su propio `update`, `events` y `draw`. Para añadir una nueva pestaña (audio, controles, etc.) basta con crear la clase e insertarla en el diccionario `self.TABS` sin modificar `SettingsScene`.
- **Pending / Commit:** los cambios se acumulan en `ResourceManager._pending` y solo se vuelcan al `ConfigParser` vivo mediante `apply_pending()`, que se invoca justo antes de escribir `config.ini` en `Game.quitGame()`.

1.2.9.3. Detalles de implementación

Cambios pendientes: Al pulsar `Apply`, el método `_apply()` calcula la nueva escala automáticamente a partir de la resolución elegida y registra todos los valores como pendientes:

```
def _apply(self):
    w, h = vid.resolution  # o resoluci n nativa si fullscreen
    ResourceManager.set_pending("video", "xres",       str(w))
    ResourceManager.set_pending("video", "yres",       str(h))
    ResourceManager.set_pending("video", "fullscreen", "1" if vid.
        fullscreen else "0")
    ResourceManager.set_pending("video", "maxfps",     str(vid.maxfps
        ))
```

```
    ResourceManager.set_pending("video", "scale",      str(min(w //
        rw, h // rh)))
    self._pending = True
```

El flag `self._pending` activa un mensaje "Reinicia para aplicar" en pantalla, informando al jugador de que los cambios surtirán efecto en el siguiente inicio.

Selección cíclica de opciones: Los botones < y > incrementan o decrementan el índice sobre las listas de opciones con aritmética modular, evitando desbordamientos:

```
if self.res_prev.update(dt):
    self._res_i = (self._res_i - 1) % len(self.RESOLUTIONS)
if self.res_next.update(dt):
    self._res_i = (self._res_i + 1) % len(self.RESOLUTIONS)
```

1.3. Aspectos Destacables

- **Arquitectura orientada a componentes con separación clara de responsabilidades:** El jugador, los enemigos y los objetos del mundo no son clases monolíticas sino entidades que componen piezas reutilizables (Graphic, Input, Movement, Health, ChasePlayer). Esto hace que añadir un nuevo tipo de entidad sea cuestión de combinar componentes existentes sin duplicar lógica. La misma clase Graphic gestiona la animación del jugador, la luz que lleva encima, los corazones del HUD y los tiles del mapa.

- **Sistema de escenas con pila (sceneStack):** En lugar del patrón habitual de una única escena activa, el motor implementa una pila que permite apilar escenas sin destruir las anteriores. Esto habilita los overlays (pausa, game over, diálogo) de forma natural: la escena subyacente sigue en memoria y se dibuja como fondo, sin necesidad de serializar ni reconstruir su estado. Es una solución elegante que muchos motores amateur no implementan correctamente.

- **Sistema de iluminación por multiplicación de capas:** La oscuridad del juego no es un simple filtro oscuro encima de la pantalla. Se construye renderizando todas las fuentes de luz sobre una superficie negra con BLEND_RGBA_ADD y multiplicando el resultado sobre la escena con BLEND_MULT, produciendo una oscuridad real donde la luz no llega. Cada fuente de luz queda además recortada al rect de su habitación para evitar que se filtre a través de las paredes, lo que requiere mantener una lista de rooms con sus coordenadas en mundo.

- **Carga de niveles completamente data-driven desde Tiled:** Los objetos del juego — jugador, enemigos, interruptores, puertas, luces, habitaciones — no están hardcodeados en el código sino definidos en el fichero .tmx mediante tipos y propiedades. El método _load_from_tiled los instancia dinámicamente por tipo y conecta automáticamente los observadores leyendo la propiedad target de cada switch. Añadir un nuevo nivel o modificar la disposición de enemigos y puzzles no requiere tocar una sola línea de Python.

- **Patrón Observer para la lógica de puzzles:** La conexión entre interruptores y puertas se implementa mediante el patrón Observer clásico (Observable / Observer), desacoplando completamente al emisor del receptor. Un switch puede notificar a

múltiples puertas a la vez, y las puertas no saben nada del switch que las activó. Esto hace el sistema de puzzles extensible sin modificar las clases existentes.

- **Sistema de recursos con caché centralizada:** El ResourceManager actúa como caché global de assets: imágenes, atlases de sprites, mapas, fuentes, sonidos y configuración se cargan una sola vez y se reutilizan por referencia. Esto evita cargas duplicadas y centraliza toda la gestión de rutas de ficheros en un único punto, facilitando el cambio de estructura de directorios sin afectar al resto del código.
- **Renderizado por capas con Y-sorting:** Las entidades no se dibujan en orden de inserción sino ordenadas por su coordenada Y inferior en cada frame, simulando correctamente la profundidad en perspectiva cenital: una entidad más abajo en pantalla tapa a una que está más arriba. El mapa de primer plano (techos, cubiertas) se dibuja después de las entidades, permitiendo que el jugador quede oculto bajo estructuras del escenario.
- **Logger de depuración no intrusivo:** El DebugLogger usa el módulo estándar logging con un RotatingFileHandler que limita el tamaño del log a 1 MB con rotación automática. Cuando el debug está desactivado en config.ini, el logger es None y cada llamada a log() es un simple check de if cls._logger, sin overhead apreciable en producción.

1.4. Manual de Usuario

1.4.0.1. Requisitos previos. El juego está desarrollado en Python 3 y únicamente requiere la librería `pygame` y `pytmx`. Para instalarlas, ejecuta en la terminal:

```
pip install pygame pytmx
```

Listing 1.1: Instalación de dependencias

1.4.0.2. Ejecución.

```
python main.py
```

Listing 1.2: Arranque del juego

El juego leerá automáticamente el archivo de configuración `config.ini` situado en el mismo directorio. No es necesario ningún argumento adicional.

1.4.1. Menú principal

Al iniciar el juego aparece el **menú principal** con tres opciones seleccionables con el ratón:

- **Play** — inicia una nueva partida (o retoma la última guardada si existe).
- **Settings** — abre el menú de ajustes de vídeo.
- **Quit** — cierra el juego guardando la configuración.

Figura 1.1: Pantalla de menú principal

Tecla	Acción
`W`	Mover hacia arriba
`S`	Mover hacia abajo
`A`	Mover hacia la izquierda
`D`	Mover hacia la derecha
`E`	Interactuar con un interruptor (al estar cerca)
`G`	Guardar partida
`Escape`	Abrir / cerrar el menú de pausa
`Espacio` / `Enter`	Avanzar diálogo

Cuadro 1.2: Controles del juego

1.4.2. Controles

Los controles de movimiento pueden modificarse editando los valores de la sección `[controls]` en `config.ini` (los valores son códigos de tecla de pygame).

1.4.3. Mecánicas de juego

- **Movimiento y colisiones**

  El jugador se desplaza en las cuatro direcciones cardinales. Las paredes y zonas infranqueables del mapa bloquean el movimiento automáticamente; no es posible atravesarlas.

- **Interruptores y puertas**

  A lo largo del mapa encontrarás **interruptores** señalizados visualmente. Acércate a uno y pulsa `E` para activarlo o desactivarlo. Cada interruptor está vinculado a una o varias **puertas**: al activarlo, la puerta se desbloquea; al desactivarlo, vuelve a bloquearse.

  Las puertas desbloqueadas se abren automáticamente cuando el jugador se aproxima a ellas y se cierran al alejarse.

- **Enemigos**

  Las criaturas llamadas **Shadows** patrullan el mapa. Si te acercas a su rango de visión, comenzarán a perseguirte. El contacto con un Shadow inflige daño al jugador.

- **Sistema de salud**

  El jugador dispone de **3 puntos de vida**, representados como corazones en la esquina superior izquierda de la pantalla. Al recibir daño, se pierde medio corazón. Si la vida llega a cero, aparece la pantalla de *Game Over*.

- **Diálogos**

  En determinados momentos del juego se mostrarán cajas de diálogo en la parte inferior de la pantalla con un efecto de texto progresivo (*typewriter*). Pulsa `Espacio` o `Enter` para:

  - Mostrar el texto completo de golpe si aún se está escribiendo.
  - Pasar al siguiente mensaje una vez mostrado.

1.4.4. Guardar y cargar partida

Pulsa `G` durante el juego para guardar la partida. El archivo de guardado se almacena automáticamente en la carpeta `../User/saved_game.json`.

Al pulsar **Play** desde el menú principal, la partida guardada se carga automáticamente, restaurando la posición del jugador y el estado de los objetos del nivel.

1.4.5. Menú de pausa

Pulsa `Escape` durante el juego para abrir el menú de pausa. Las opciones disponibles son:

- **Resume** — vuelve al juego en el punto exacto donde se pausó.
- **Settings** — accede a los ajustes de vídeo.
- **Quit** — abandona la partida y vuelve al menú principal.

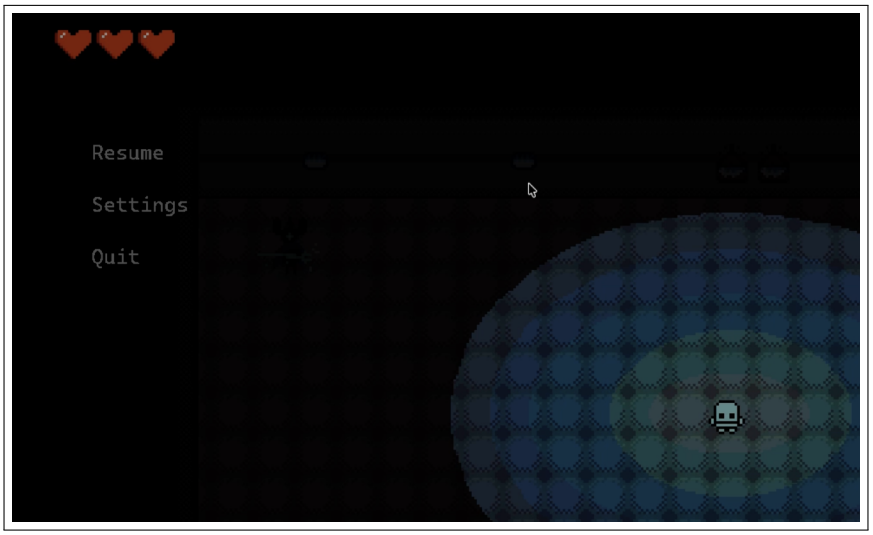


Figura 1.2: Pantalla de Pausa

1.4.6. Pantalla de Game Over

Si el jugador muere, la pantalla se tine de rojo y aparecen dos opciones:

- **Retry** — restaura la vida del jugador y recarga el último guardado.
- **Main Menu** — vuelve al menú principal.

1.4.7. Ajustes de vídeo

Accesible desde el menú principal o el menú de pausa. Permite modificar:

- **Resolución** — cicla entre las resoluciones disponibles con los botones `<` y `>`.
- **Pantalla completa** — activa o desactiva el modo fullscreen con el botón `Toggle`.
- **FPS máximos** — ajusta el límite de fotogramas por segundo.

Pulsa **Apply** para confirmar los cambios. Los ajustes se aplicarán en el siguiente inicio del juego. Pulsa **Back** para volver sin guardar.

Figura 1.3: Ajustes de vídeo

1.5. Reporte de *Bugs*

BUG-01 **Código muerto en `GameScene.update()`.** El bloque de las líneas 76–93 (actualización de entidades por rooms) nunca se ejecuta: está indentado dentro del bloque `if not self.player: return`, por lo que el intérprete lo trata como parte del `return` y lo ignora. El culling por rooms solo funciona en el segundo bloque (líneas 108–123). Eliminar el bloque muerto o, si la intención era unificar el culling de shadows y entidades, reescribir el método con un único bucle que itere primero los shadows y luego el resto, sin duplicar lógica.

BUG-02 **El estado del jugador no se persiste completamente al guardar.** `Player.serialize()` solo guarda la posición (`pos.topleft`). El inventario de llaves

(`self.keys`), la vida actual (`health.current_hp`) y la dirección que mira el personaje (`facing`) no se guardan. Si el jugador recoge una llave, pierde vida y guarda, al recargar la llave reaparece y la vida se restablece. Ampliar `serialize()` para incluir `keys`, `health.current_hp` y `facing`. Ampliar `unserialize()` para restaurar esos valores, teniendo cuidado de sincronizar `Movement._x` y `Movement._y` tras cambiar `pos.topleft`.

BUG-04 **`Key` no implementa `serialize` ni `unserialize`.** La clase `Key` hereda de `abstract.Object`, que declara ambos métodos como obligatorios y lanza `NotImplementedError` si no están definidos. Al intentar guardar la partida con una llave en la escena, `SaveManager.save()` llama a `ent.serialize()` sobre la llave y el juego lanza una excepción. Implementar `serialize()` devolviendo `{ïs_active": self.is_active}` e `unserialize(data)` restaurando `self.is_active` y eliminando el sprite del grupo gráfico si la llave ya fue recogida.

BUG-05 **Doble recogida de llave por colisión y tecla `P` simultáneas.** `Key.on_collision()` llama a `interact()` directamente. `Key.update()` también llama a `interact()` si se pulsa `P` en rango. Si el jugador está en contacto con la llave y pulsa `P`, se ejecutan ambas rutas en el mismo frame: `interact()` se llama dos veces, añadiendo la llave al inventario dos veces y emitiendo `KEY_PICKED` dos veces (desbloqueando la puerta dos veces, lo que podría volver a bloquearla si la puerta reacciona al evento). Añadir al inicio de `interact()` una comprobación `if not self.is_active: return` (la llave ya se desactiva al final del método, pero la segunda llamada ocurre antes de que el estado se actualice). Alternativamente, eliminar la lógica de recogida por tecla `P` de `update()` y unificarla exclusivamente en `on_collision()`.

BUG-06 **La puerta no tiene `serialize` ni `unserialize`.** `Door` hereda de `abstract.Object` pero no implementa los métodos de serialización. Al guardar, `SaveManager` iterará las entidades del grupo e intentará llamar a `door.serialize()`, obteniendo `NotImplementedError`. El estado de la puerta (bloqueada, abierta) no se persiste entre sesiones. Implementar `serialize()` devolviendo `{ïs_locked": self.is_locked, ïs_open": self.is_open}` e `unserialize(data)` restaurando ambos flags y actualizando el estado gráfico correspondiente.

BUG-07 **Importaciones duplicadas en `scenes.py`.** Las líneas 7 y 9 importan `LightObject` y `Portal` dos veces desde `objects`. Además, en el diccionario `entity_classes` de `_load_from_tiled()` la entrada `"Table": table.Table` aparece duplicada (líneas 191–192). Aunque no produce un error en ejecución, genera confusión y la segunda entrada simplemente sobreescribe la primera en el diccionario. Eliminar la importación duplicada de la línea 9 y una de las dos entradas `"Table"` del diccionario.

BUG-08 **El jugador puede moverse en diagonal pero la animación solo refleja un eje.** En `Player.update()`, la selección de animación evalúa primero el eje Y y solo comprueba el eje X en un `elif`, por lo que si el jugador se mueve en diagonal (por ejemplo, abajo-derecha) siempre se muestra la animación de bajar, nunca la de moverse a la derecha. El movimiento físico sí se produce en ambos ejes. Definir la prioridad de animación explícitamente (por ejemplo, dar prioridad al eje de mayor magnitud) o, dado que `Input.update()` normaliza el vector, evaluar el signo de cada componente con una condición prioritaria acordada por el equipo.